8-Bit Linear Phase FIR Filter (Part 1)

Devansh Modi

Abstract—In this assignment, we aim to compare Array Multipliers and Wallace Tree Multipliers. We start with the analysis of the 5x5 Array and Wallace Tree Multipliers. Based on the simulation results and mathematical calculations, we will prove that the Wallace Tree Multiplier is a better architecture than the Array Multiplier in terms of Performance, Area, and Power. Then we generalize this observation for the NxM multiplier. Then we move on to an MxN Wallace Tree multiplication, where (M ≠ N), by taking an example of a 5x4 Multiplier. We try to find out which one is a better way to implement, i.e., 5x4 or 4x5 or 5x4 using 5x5. Finally, we will show for which applications the two different Multipliers should be used.

I. INTRODUCTION

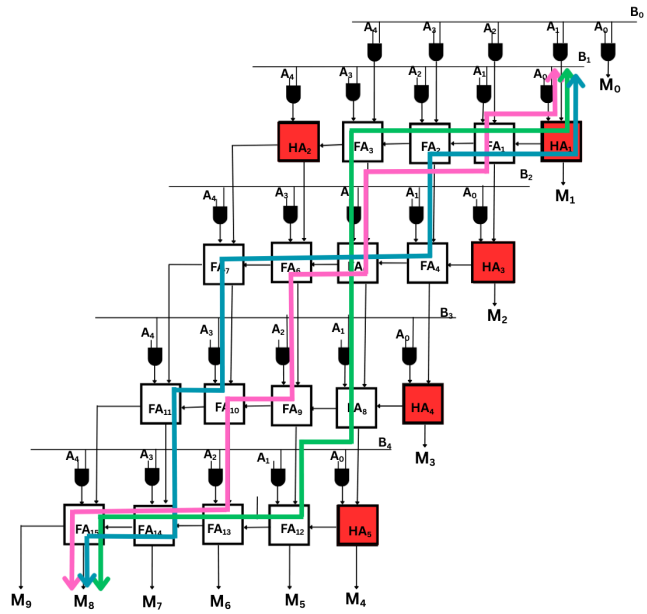
For this assignment, we are going to use typical 0.18 µm CMOS delays as a reference. For example, FO4 loads at 180 nm give an inverter delay ≈60 ps and a 2-input NAND ≈72 ps . Using these as a baseline, we estimate the following propagation delays at 1.8 V, for a "standard" load (roughly FO4):

- **2-input NAND** ≈ 72 ps (0.072 ns)
- **2-input AND** ≈ 72 ps + 60 ps ≈ 132 ps (0.13 ns)
- **3-input AND** ≈ 150 ps (0.15 ns)
- **4-input AND** ≈ 180 ps (0.18 ns)
- **2-input XOR** ≈ 240 ps (0.24 ns)
- **2-input OR** ≈ 90ps (0.09 ns)
- **3-input OR** ≈ 118 ps (0.118 ns)

All values above are approximate and assume typical (FO4) loading at 1.8 V. They are derived from known 180 nm CMOS characterizations (e.g. FO4 INV=60 ps, NAND2=72 ps) and logical effort scaling. Exact rising vs. falling delays are device-dependent; the numbers given are rough averages. The HA's and FA's are the basic building blocks of these multipliers. As we will see in the later section, critical path delay can be reduced by balancing the Sum and Carry delay. But, as we are building the adder structure using gates we would not be able to balance the Sum and Carry delay. To balance the delays we have to build these adders at the transistor level which we are not doing for this assignment.

II. 5x5 Array Multiplier

The below figure shows the diagram of a 5x5 Array Multiplier. Whenever only two bits are left or available to combine in a certain position a half adder is used instead of a full adder. Since half adders are much simpler and smaller than full adders, this reduces the total area of the multiplier.



(Fig. 1. 5x5 Array Multiplier)

The delay of array multiplier (balanced Sum and Carry Stages) can be given by,

$$T_{ARRAY} = T_{AND} + (N + M - 3)T_{CARRY} + (M - 1)T_{SUM}$$

From the above figure we can see that there are multiple critical paths. The colored paths show different critical paths in the figure 1. This is typical of generally-sized multipliers. Notice that for very large multipliers, or if FAs are used everywhere instead of HAs, all paths that lead from A_0B_0 to the penultimate M-bit become critical paths. Multiple paths with the same slack at the same supply mean that the circuit has little room for improvement. Trying to improve the speed of a combinational block involves trying to reduce the propagation delay of the critical path. Because the multiplier has multiple equal paths, improving any one of them would cause the critical path to migrate to the others.

If the delay of Sum and Carry stages is not balanced than the critical path delay can be given by,

$$T_{ARRAY} = T_{AND} + (N + M - 3)T_{CARRY} + K(M - 1)T_{SUM}$$

where K = ratio of Sum and carry delay which is greater than 1.

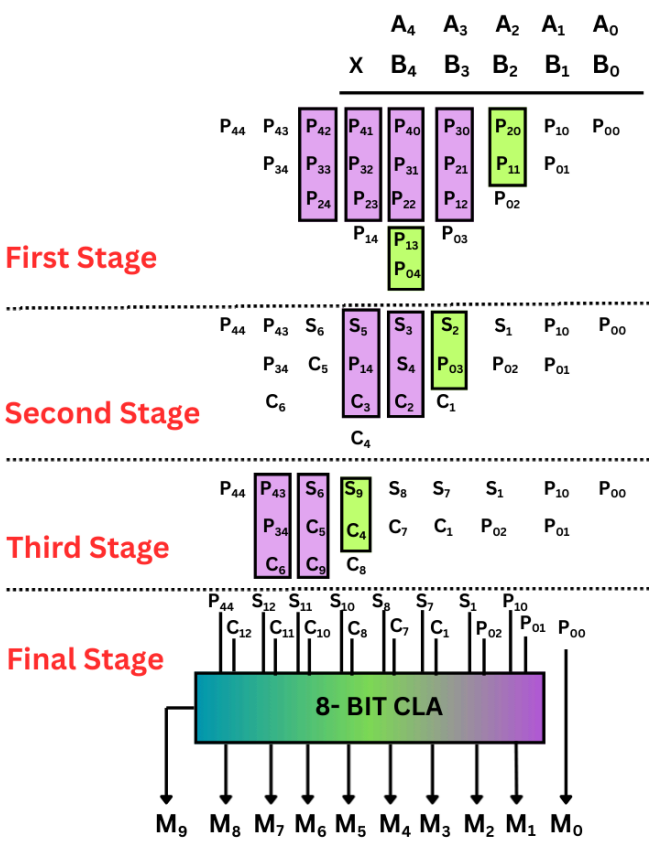
Clearly, from the two equations we can see that we can optimize the Array Multiplier by balancing the Sum & Carry stage of the Adder.

After doing the simulations critical path delay for the 5x5 Array Multiplier was found to be **2.808 ns** and the maximum possible clock frequency will be **356 MHz**.

Trying to optimize all paths at the same time is normally paid for in terms of loading on previous stages and total chain effort. Thus, sizing cannot generally be used to speed up array multipliers. But improving upon multiplier speed is very important, because multipliers tend to dominate the critical path. This can only be done by changes in the algorithmic or architectural level. This can be done by improving the procedure for combining partial products as in Wallace tree.

III. 5x5 Wallace tree multiplier

The diagram of the 5x5 Wallace tree is shown below, the green box is HA and purple box is FA.



(Fig. 2. 5x5 Wallace Tree Multiplier)

If we assume that the Sum & Carry stages are balanced than in each stage (except the final stage), the output from all the adders used in that stage will be generated simultaneously. All the input bits in the final stage will arrive at the same time. This is in contrast to the Array Multiplier where all the input bits were not coming simultaneously to the last stage. Another thing to note is that there are not multiple critical paths in Wallace Tree. There is a unique critical path and the final vector addition is on that critical path which can be optimized.

This is the first advantage of the Wallace Tree Multiplier. We can implement a fast vector addition in the final stage as all the input bits to that stage are coming simultaneously and the last stage is on the critical path. For an Array Multiplier we can not do this optimization. In 5x5 Multiplication we are using a CLA as there are 8 bits in the final vector addition stage. If the no. of bits are more in the final stage, then we can implement other adder architectures like carry skip or carry select.

The area of Wallace Tree Multiplier will slightly increase because of the fast adder architecture in the final stage. But we will get a significant improvement in performance.

As seen in Array Multiplier, it is necessary to balance the Sum and Carry stages in Wallace Tree. If they are not balanced then the outputs from the intermediate stages will not be simultaneous. This will increase the critical path delay. The critical path delay of the 5x5 Wallace Tree multiplier can be given by the equation,

$$T_{WALLACE} = T_{AND} + 3.T_{SUM} + T_{MERGE}$$

where T_{MERGE} is the delay of final vector addition.

After doing the simulations critical path delay for the 5x5 Array Multiplier was found to be **2.438 ns** and the maximum possible clock frequency will be **410 MHz**. The difference between the delay of two multipliers is **370 ps**. Here for small no. of bits the difference is not too satisfactory. But, if we implement Wallace Tree Multiplication for a large number of bits then the difference would be significant.

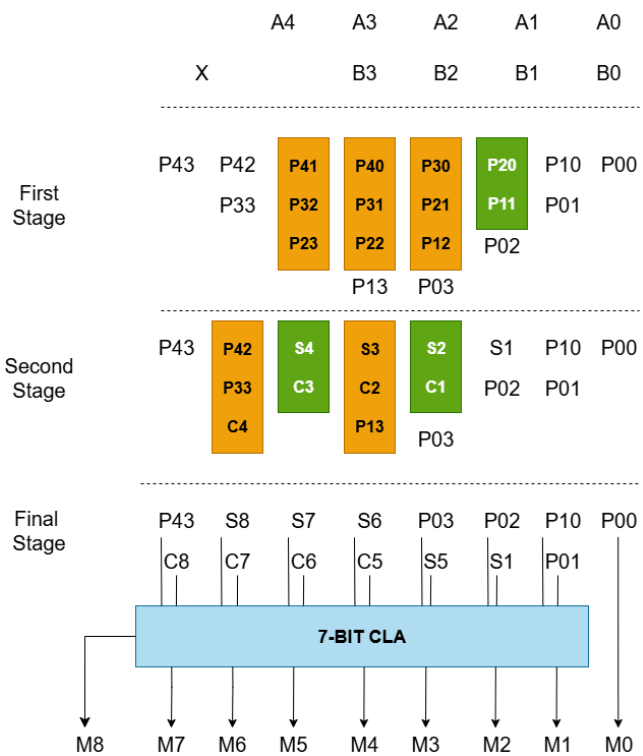
IV. 5x4 Wallace tree Multiplier

In this section we will analyze a N x M multiplier where $M \neq N$. We take N = 5 & M = 4. There are different ways to implement it. We can implement 5x4, 4x5 or 5x4 using 5x5. The Wallace tree for 5x4 & 4x5 will be the same and therefore there is no difference if we implement 5x4 or 4x5. In Array Multipliers we can reduce the number of stages by implementing 5x4 as compared to 4x5.

Figure 3 shows a 5x4 Wallace Tree Multiplier. Green colour represents HA and orange colour represents FA. The delay of this multiplier can be given by the equation,

$$T_{WALLACE(5x4)} = T_{AND} + 2.T_{SUM} + T_{MERGE}$$

where T_{MERGE} is the delay of final vector addition.



(Fig. 3.) 5x4 Wallace Tree Multiplier

After simulating, critical path delay for the 5x4 Wallace Tree Multiplier was found to be **2.288 ns** and the maximum possible clock frequency will be **437 MHz**. If we do it using a 5x5 Wallace Tree multiplier the delay is found to be **2.39 ns**. We can observe from the above simulation results that 5x4 Multiplier results in lesser delay than 5x5 multiplier. Also the number of FA's & HA's used are less in 5x4 multiplier reducing the area of overall multiplier. We can generalize this by saying that if in our application we are doing a specific multiplication only, i.e., NxM multiplication where M $\neq$ N, than it is better to make a NxM multiplier architecture rather than making max(NxM)x max(NxM) architecture.

V. Advantages and disadvantages

Wallace-tree multipliers are faster than array multipliers because they reduce partial products in parallel using carry-save adders, achieving logarithmic delay (O(log n)) compared to the linear delay (O(n)) of arrays. This makes Wallace trees ideal for high-speed, low-latency applications like CPUs, GPUs, and DSPs, though they come with higher design complexity, irregular layout, and often larger area. Array multipliers, on the other hand, are simple, regular, and easy to implement, but slower and more power-hungry due to long carry chains; they are better suited for small designs, FPGAs, or cost-sensitive systems. In short, Wallace trees trade area and complexity for speed, while arrays trade speed for simplicity and regularity.

VI. Conclusion

In this assignment, we compared the performance of Array and Wallace Tree Multipliers using 5×5 and 5×4 configurations. Simulation results showed that the 5×5 Wallace Tree Multiplier achieved a lower critical path delay of 2.438 ns and a higher maximum clock frequency of 410 MHz, compared to the 5×5 Array Multiplier with a delay of 2.808 n**s** and frequency of 356 MHz. These results demonstrate that the Wallace Tree architecture is more efficient in terms of speed and timing, especially for larger multipliers, due to better optimization of partial product reduction and critical path balancing. Then we showed that if, in any hardware design, it is known that only 5×4 multiplication is required, then it is better to implement a 5×4 Wallace Tree Multiplier directly. Using a 5×5 multiplier for 5×4 operations would unnecessarily increase area, add delay, and reduce overall performance.

VII. References

[1] Handbook of Digital CMOS Technology, Circuits, and Systems - Karim Abbas.

[2] CMOS VLSI Design: A Circuits and Systems Perspective - Book by David Harris and Neil Weste.

[3] Introduction to Digital Systems: Milós Ercegovac, Tomás Lang, Jaime H. Moreno